

2. Introduction to Distributed System

Q.1 Explain different issues and goals related to design of distributed system.

⇒

- A distributed system is defined as a set of autonomous computers that appears to its users as a single coherent system.
- Users of distributed system feel that, they are working with a single machine.
- Following are the issues and goals related to design of distributed system:-

- 1) Heterogeneity
- 2) Making resources accessible
- 3) Transparency
- 4) Scalability
- 5) Openness
- 6) Security
- 7) Reliability

1) Heterogeneity

⇒ We cannot have homogeneous environment in wide area network.

- The type of networks, architectures, operating systems, programming languages cannot be expected to be same.
- Internet enables users to access services and run applications in heterogeneous environment.
- For e.g. a computer attached to an Ethernet has an implementation of IP over the Ethernet, whereas a computer on different

sort of network will need an implementation of the IP for that network.

2) Making Resources Accessible

- ⇒ Users and applications should be able to access the remote resources in easy way.
- Distributed system should allow sharing these remote resources in efficient and controlled manner.
- Resource sharing offers saving in cost. One printer can be shared among many users in office instead of having one printer to each individual user.
- There can be more saving in cost if expensive resources are shared.

3) Transparency

⇒ The main goal of distributed system is to hide the actuality of physical distribution of processes and resources across several computers.

a) Access transparency: It hides differences in data representation and storage, allowing users to interact with resources uniformly across systems.

b) Location transparency: Location transparency makes the physical location of resources invisible, enabling users to interact with resources uniformly across systems.

c) Replication Transparency: It hides the replication of data across multiple nodes, allowing users to access it as if there were a single copy.

4) Scalability

⇒ Distributed systems operate effectively and efficiently at many different scales, ranging from a small intranet to the Internet.

- A system is described as scalable if it will remain effective when there is a significant increase in the number of resources and the number of users.

5) Openness

⇒ The openness of computer system is the characteristic that determines whether the system can be extended and re-implemented in various ways.

- The openness of distributed systems is determined primarily by the degree to which new resource-sharing services can be added and be made available for use by a variety of client programs.

6) Security

⇒ Many of the information resources that are made available and maintained in distributed systems have a high intrinsic values to their users.

- Security for information resources has three components:

a) Confidentiality: Protection against disclosure to unauthorized individuals.

b) Integrity: Protection against alteration or corruption.

c) Availability: Protection against interferences with means to access the resources.

7) Reliability: ...

⇒ By distributing tasks across multiple nodes, system becomes more fault-tolerant.

- Even if one node fails, others can continue functioning, ensuring uninterrupted service.

8) Performance ...

⇒ Efficient resource allocation and load balancing lead to improved performance.

- Tasks can be parallelized, reducing execution time.

Q.2 Discuss in brief the different architectural models in 'Distributed System'.

⇒

1. Models.

- 1) Physical Model
- 2) Architectural Model
- 3) Fundamental Model

1) Physical Model

⇒ A physical model is a representation of the underlying hardware elements of a distributed systems.

- It captures the hardware composition of a system in terms of computers and other devices and their interconnecting network.

- The main components in physical model :-

a) Nodes : These are the end devices that can process data, execute tasks, and communicate with other nodes.

b) Links : These are the communication channels between different nodes and intermediate devices.

c) Middleware : These are the software installed and executed on the nodes.

d) Network Topology : This defines the arrangement of nodes and links in the distributed computing system.

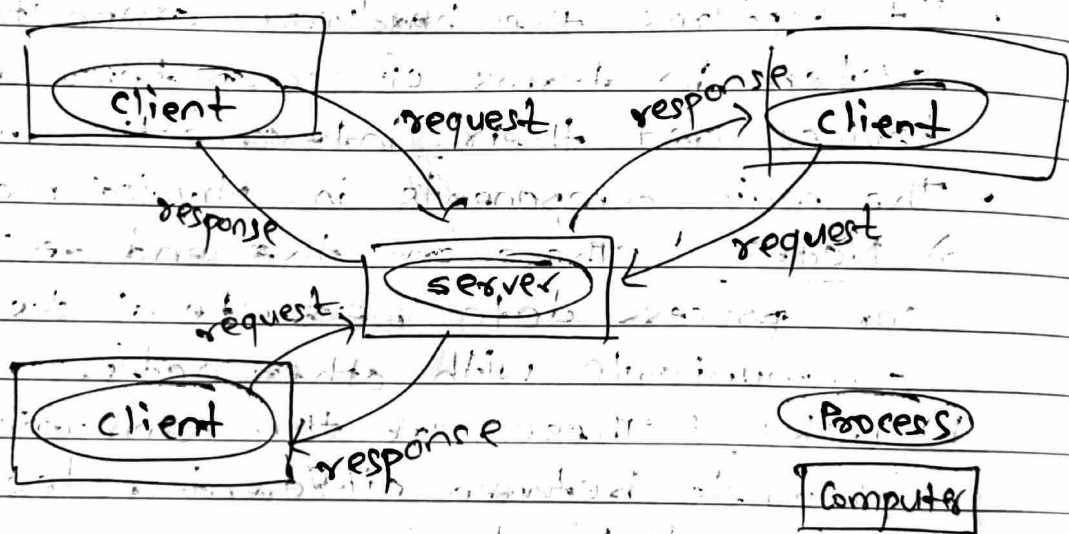
2) Architectural Model

⇒ Architectural model defines the way in which the components of the system interact with each other and the way in which they are mapped onto an underlying network of computers.

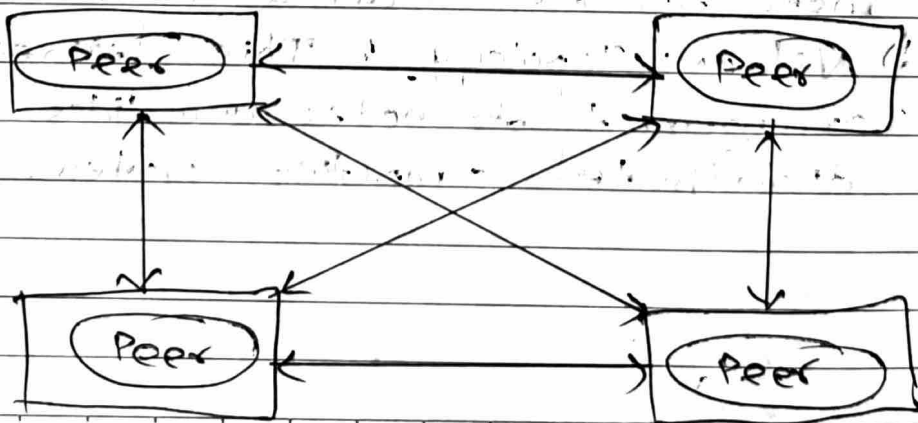
It simplifies and abstracts the functionality of individual components of a distributed system.

For e.g., Client-server Model and Peer-to-Peer Model.

a) Client-Server Model



b) Peer-to-Peer Model



3) Fundamental Model

- ⇒ The purpose of fundamental model is to make explicit all relevant assumptions about the system we are modeling.
- It includes interaction, fault and security model.

a) Interaction Model: Interaction model deals with performance and are used for handling time in distributed systems.

b) Failure Model: This model addresses the faults and failure that occurs in distributed computing system. It provides a framework to identify and rectify the faults that occur or may occur in the system.

c) Security Model: Distributed computing systems may suffer malicious attacks, unauthorized access and data breaches. Security model provides a framework for understanding the security requirements, threats, vulnerabilities to safeguard the system and its resources.

Q.3 What are services offered by middleware?

- ⇒
- Middleware acts as a software that acts as bridge between applications and operating systems in distributed computing environment.
 - It enables communication, coordination and management of distributed applications.
 - Services offered by Middleware

- 1) Communication services
- 2) Transaction management
- 3) Security services
- 4) Resource management
- 5) Directory services

1) Communication Services

⇒ Middleware provides APIs and abstraction for communication between distributed components.

- Supports Message queuing, Data streaming, and Protocol translation.
- Facilitates SOAP and REST API communication for web services.
- For e.g., Apache Kafka, ZeroMQ

2) Transaction Management

- ⇒ Ensures Atomicity, Consistency, Isolation and Durability (ACID) in distributed systems.
- Uses protocols Two-phase Commit (2PC) and Three-phase Commit (3PC) for co-ordination.
 - Rollback and Recovery mechanism.

- For e.g., Microsoft transaction server, JTA

3) Security services

⇒ Middleware provides secure communication and data access.

- Provides features like authentication, authorization and data encryption.
- Ensures secure data transmission through TLS/SSL and AES encryption.
- For e.g., Kerberos, OAuth.

4) Resource management

⇒ Efficiently allocates and manages distributed systems resources like CPU, memory and storage.

- Implements load balancing to distributed traffic across multiple servers.
- For e.g., Kubernetes, Apache Zookeeper.

5) Directory services

⇒ Provides a centralized directory for resource and service discovery.

- Implements naming and lookup services.
- Supports service registration and discovery for locating services dynamically.
- For e.g., LDAP, DNS.

Q.4. Differentiate between NOS, DOS and Middleware in design of distributed system.

⇒

NOS	DOS	Middleware
1) Provides local services to remote clients, managing resources among connected system.	1) Manages hardware resources collectively across all nodes, offering cohesive resource management approach.	1) Acts as layer between operating system and applications.
2) High scalability, allowing addition of more system without significant performance degradation.	2) Less scalable, as adding nodes can introduce complexity and potential performance degradation.	2) Middleware itself does not directly affect scalability but enables scalable communication across different systems.
3) Lower fault tolerance, less capable of continuing operation in event of a failure.	3) Higher fault tolerance, better equipped to handle failures without disrupting overall system operation.	3) Middleware facilitates fault tolerance by providing reliable messaging and data services.
4) Same operating system is present on all nodes in both cases.	4) OS on different nodes is different.	4) OS on different nodes is different.

NOS	DOS	Middleware
4) Operating system on different nodes is different.	4) Same OS is present on all nodes.	4) OS on different nodes is different.
5) Higher rate of autonomy, with each node operating independently large extent.	5) Lower rate of autonomy, indicating a more unified control over the entire network.	5) Middleware facilitates interaction between systems but does not dictate autonomy.
6) Based on Files, Facilitating the sharing and management of resources.	6) Based on messages and shared memory, allowing for integrated resource management.	6) Provides communication, data exchange services, including RPCs and message queues.
7) Complex for developers.	7) Easier for developers.	7) Simplifies development by providing abstraction.
8) Lower design complexity.	8) High complexity due to tight coupling.	8) Moderate complexity.
9) E.g., Windows server	9) E.g., Amoeba, Mach	9) E.g., CORBA, JAVA RMI.

Q.5 Explain various types of Middlewares.

⇒ 1) Message-oriented Middleware (MOM)

⇒ Facilitates communication between distributed applications through message queues.

- Uses asynchronous messaging, allowing sender and receiver to operate independently.
- Ensures reliable message delivery, even in case of failures.
- Supports publish-subscribe and point-to-point messaging.
- For e.g., Apache Kafka, IBM MQ, etc.

2) Object-oriented Middleware

⇒ Enables communication between objects in distributed environments.

- Uses object request brokers (ORBs) to manage object interactions.
- Allows objects written in different languages to communicate.
- For e.g., CORBA, JAVARMI, DCOM.

3) Service-oriented Middleware

⇒ Supports service-oriented architecture by enabling seamless interaction between services.

- Provides service discovery, composition and orchestration.
- Ensures loose coupling between applications for flexibility.
- For e.g., Apache CXF, Oracle SOA Suite.

4) Database Middleware

⇒ It provides a layer between application and databases for seamless data access.

- Provides query processing, caching and connection pooling.
- Enables heterogeneous database access by supporting different database systems.
- For e.g., ODBC, JDBC, Oracle Net.

5) Remote Procedure Call (RPC) Middleware

⇒ Allows a program to execute functions on a remote server as if they were local.

- Uses synchronous communication, meaning the caller waits for a response.
- Supports cross-platform interoperability in distributed applications.
- For e.g., gRPC, Apache Thrift, JAVA RMI.

6) Event-driven ~~are~~ middleware

⇒ Supports event-based communication between distributed components.

- Uses an event bus and event-driven architecture to manage communication.
- Can integrate with serverless computing and event stream processing.
- For e.g., Apache Kafka, ActiveMQ.

Q.6 Explain Grid and Cluster computing Models.

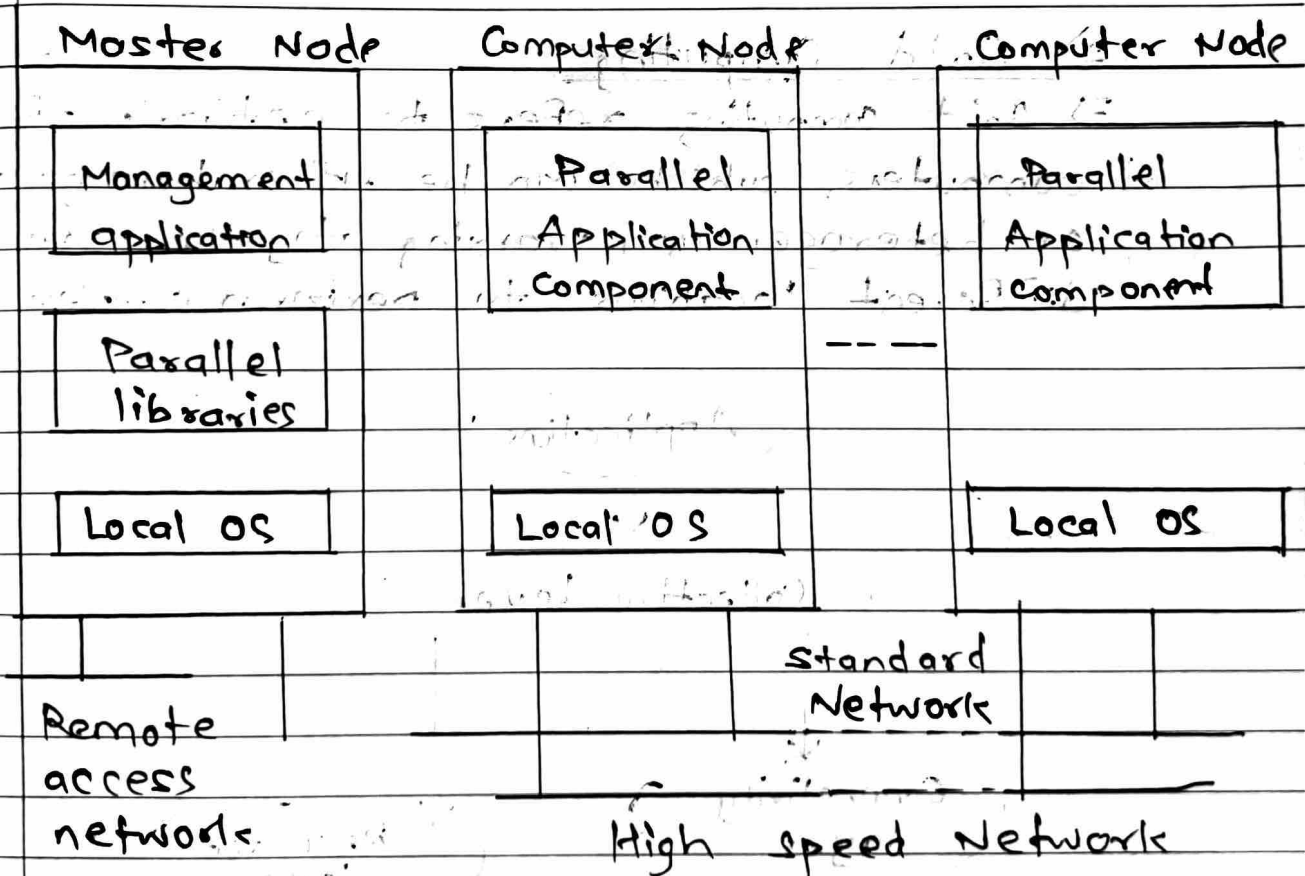
⇒

- One class of distributed system is intended for high-performance computing tasks.

1) Cluster Computing

⇒ A cluster is a group of tightly connected computers that work as a single system to perform computational tasks.

- This approach is commonly used for tasks that require high computational power and reliability.

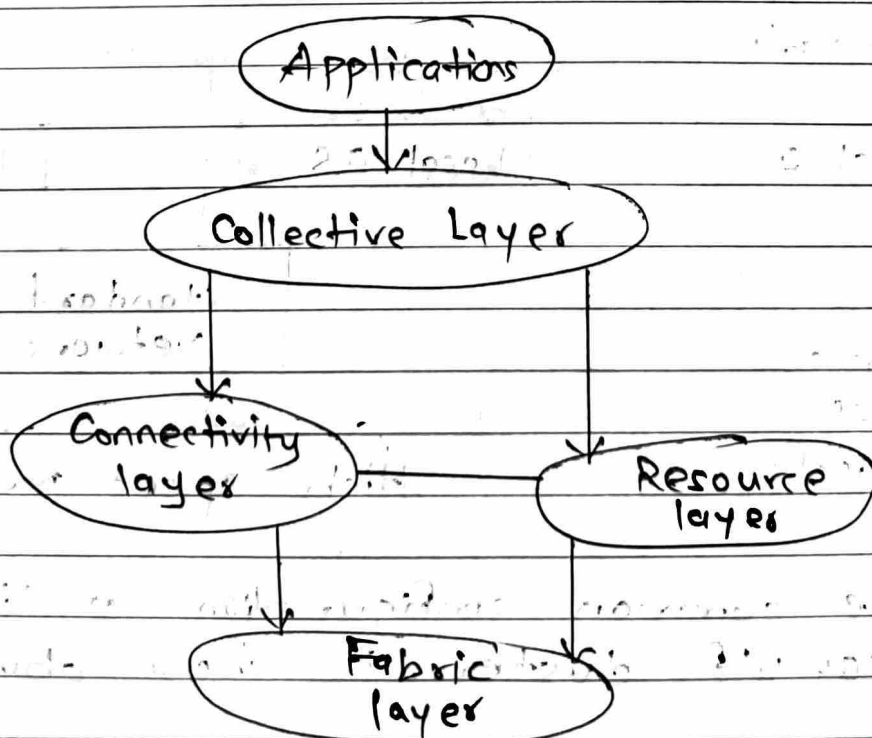


- The common configuration of Linux-based Beowulf cluster is shown above.

- Each cluster comprises a set of computer nodes.
- All these computer nodes are controlled and accessed through a single master node.
- The master node characteristically manages the allocation of nodes to a specific parallel program.
- The master in fact runs the middleware required for the execution of programs and management of the cluster, whereas the computer nodes requires only a standard OS.

2) Grid Computing

⇒ Grid computing refers to network of computers, which can be either homogenous or heterogenous, working together across different locations to perform complex tasks.



a) Fabric layer

⇒ It is a lowest layer and offers interfaces to local resources at a specific site.

b) Connectivity layer

⇒ This layer contains communication protocol to offer support for grid transactions that span the usage of multiple resources. Protocols are required to send data among servers, resources or to access a resource from a distinct location.

c) Resource layer

⇒ This layer manages a single resource. It makes use of functions offered by the connectivity layer and calls straight way the interfaces made available by fabric layer.

d) Collective layer

⇒ It handles access to multiple resources and contains services for resource discovery, allocation and scheduling tasks onto multiple resources, data replication and so on.

e) Application layer

⇒ This layer contains application that functions within a virtual organization and which utilizes the grid computing environment.

Q.7 Explain different types of distributed systems with diagrams.

⇒

Types

1) Distributed Computing Systems

- i) Cluster Computing

- ii) Grid Computing

2) Distributed Information Systems

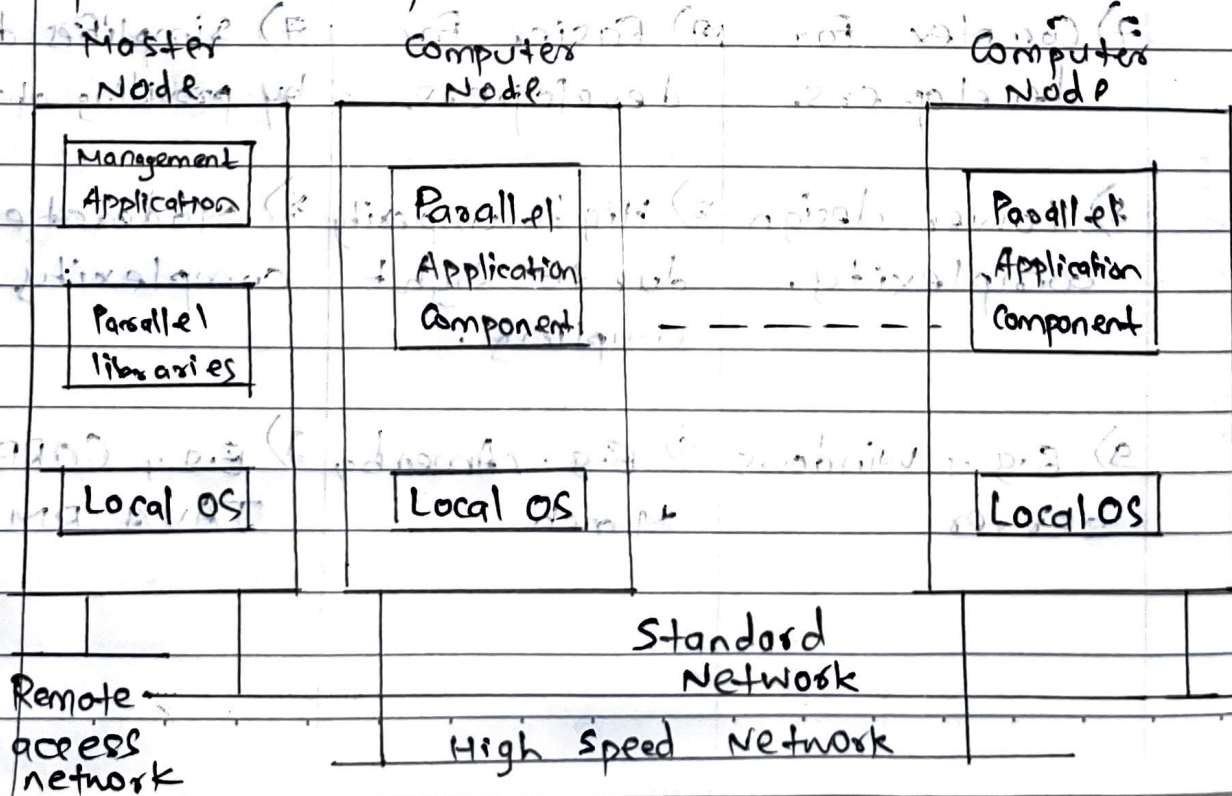
- i) Transaction Processing Systems

- ii) Enterprise Application Integration

1) Distributed Computing Systems

i) Cluster Computing

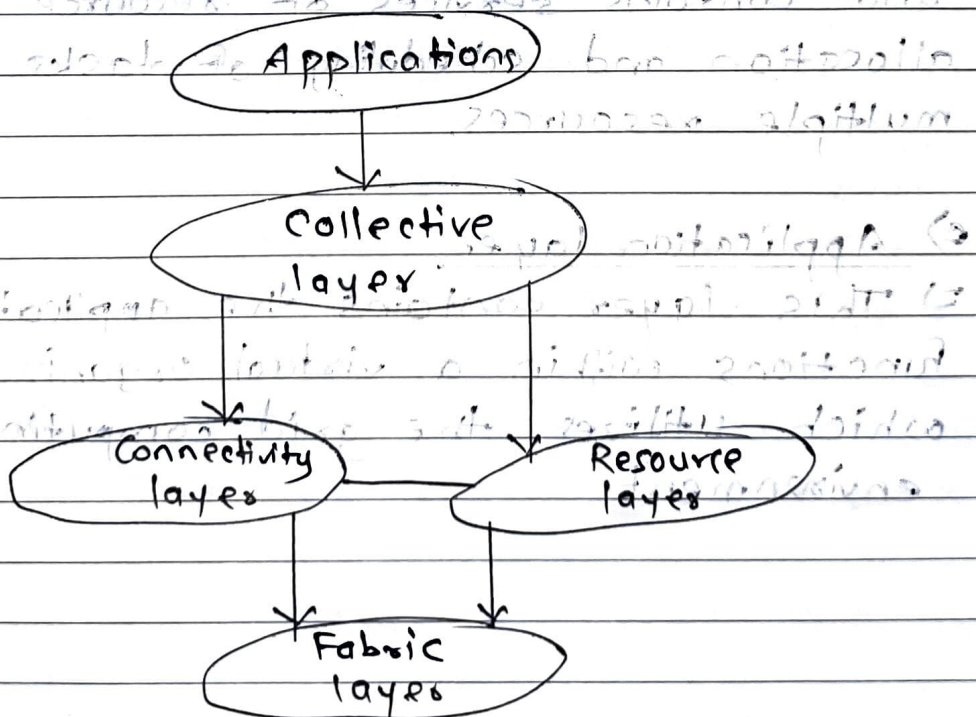
⇒ A cluster is a group of tightly connected computers that work as a single system to perform computational tasks.



- Each cluster comprises of set of computer nodes. All these computer nodes are accessed and managed by single master node.
- The master node characteristically manages allocation of nodes to specific parallel nodes.
- The master in fact runs the middleware required for the execution of programs and management of cluster.
- Whereas computer nodes requires only a standard operating system.

ii) Grid Computing

⇒ Grid computing refers to the network of computers which can be either homogenous or heterogeneous working together across different locations to perform complex tasks.



q) Fabric layer : It is the lowest layer and offers interfaces to local resources at a specific site.

b) Connectivity layer: This layer contains communication protocols that offer support for grid transactions that span usage of multiple resources. Protocols are required to send data among resources or to access a resource from distant location.

c) Resource layer: This layer manages the single resource. It makes use of functions offered by connectivity layer and calls straight way the interfaces made available by the fabric layer.

d) Fabric Collective layer

⇒ It handles access to multiple resources and contains services of resource discovery, allocation and scheduling of tasks onto multiple resources.

e) Application layer

⇒ This layer contains the applications that functions within a virtual organization and which utilizes the grid computing environment.

2) Distributed Information Systems:

i) Transaction Processing System

⇒ It provides the highly structured client-server approach for database applications.

- It obeys the ACID properties:

a) Atomicity: Execution of transaction is individual.

b) Consistent: There is no violation of system invariants by transaction.

c) Isolated: Concurrent transaction do not interfere with each other.

d) Durable: After a commit is done by transaction, the changes are permanent.

- Example primitives of transaction

a) BEGIN TRANSACTION: For making the start of transaction.

b) END TRANSACTION: For terminating the transaction and try to commit.

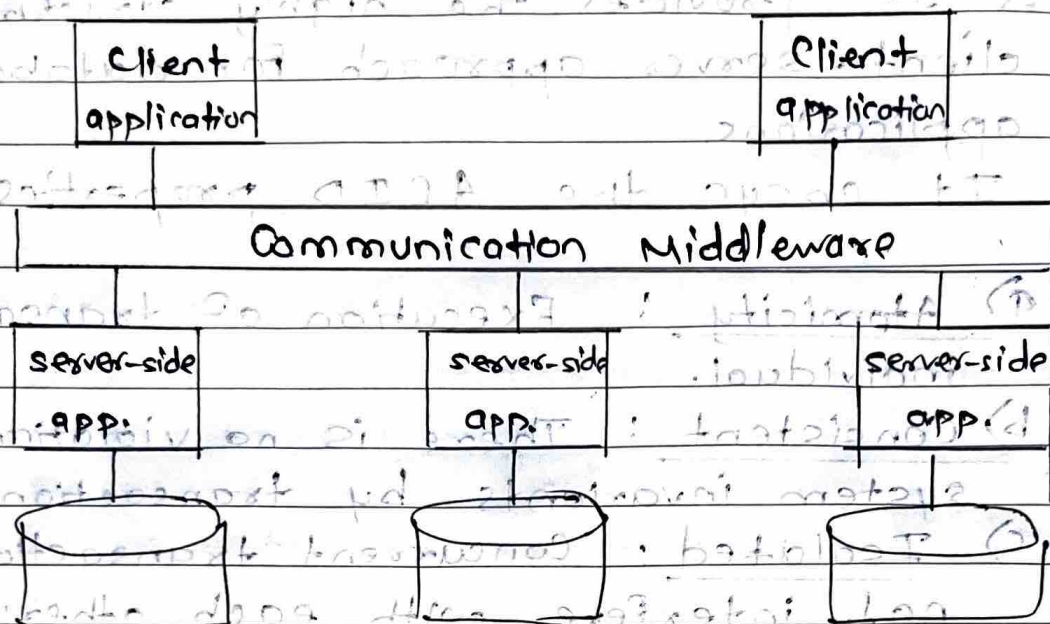
c) ABORT TRANSACTION: For killing the transaction and restore the previous state.

d) READ: To read the data from table or file.

e) WRITE: To write the data to table or file.

ii) Enterprise Application Integration (EAI)

⇒ Middleware acts as a communication facilitator in EAI.



There are many examples of communication middleware.

— Using RPC, an application component at client side can efficiently send a request to another application component at server-side by making a local procedure call.

— This request at client side is packed as a message and sent to the called application.